

RC Charging Transient -- Multiple R and C Comparison

Purpose	Simulate capacitor charging voltage $v_C(t)$ for a grid of R and C values, and show that the time constant $\tau = RC$ alone governs the shape of the response.
Supports	Module 1 (Volt-second / Current-second balance) and Module 2 (Exponential Family), EE-AOC-581/681
Compiler	Python 3.13
OS	Manjaro Linux KDE Plasma
Version	V1
Written on	2026-08-23
Revision Date	2026-08-23
Author	Kingsuk Majumdar
Copyright	(c) 2026 Kingsuk Majumdar

"Nothing is too wonderful to be true, if it be consistent with the laws of nature." -- Michael Faraday

```
In [1]: import os
import csv
from datetime import datetime

import numpy as np
import matplotlib.pyplot as plt
```

Stage 0: Output directory setup

All figures, data and reports are written to an `OUTPUT` folder next to this notebook.

```
In [2]: # __file__ is not defined inside a notebook kernel, so fall back to the
# current working directory (the folder the notebook was launched from).
try:
    script_dir = os.path.dirname(os.path.abspath(__file__))
except NameError:
    script_dir = os.getcwd()

output_dir = os.path.join(script_dir, "OUTPUT")
os.makedirs(output_dir, exist_ok=True)
print("Output directory:", output_dir)
```

Output directory: /home/kingsuk/Documents/Kingsuk_LaTeX/AddOnCourse/ClassLecture/Module_01/PythonCode/OUTPUT

Stage 1: Circuit parameter grid

A 3×3 grid of resistor and capacitor values. Change these dictionaries to explore other combinations.

```
In [3]: V_SOURCE = 5.0 # V, step input applied at t = 0

R_VALUES = {
    "R1_1k0": 1_000.0,
    "R2_4k7": 4_700.0,
    "R3_10k": 10_000.0,
}

C_VALUES = {
    "C1_100uF": 100e-6,
    "C2_220uF": 220e-6,
    "C3_470uF": 470e-6,
}
```

Stage 2: Time base

A time base of 5τ captures charging to 99.3 percent of `V_SOURCE` for the slowest (largest τ) combination in the grid.

```
In [4]: tau_max = max(R_VALUES.values()) * max(C_VALUES.values())
t_end = 5.0 * tau_max
t = np.linspace(0.0, t_end, 2000)
```

Stage 3: Compute charging curves for every R, C combination

$$v_C(t) = V_{SOURCE} \left(1 - e^{-t/RC}\right)$$

```
In [5]: results = {} # key: (R_label, C_label) -> dict with R, C, tau, v(t)

for r_label, R in R_VALUES.items():
    for c_label, C in C_VALUES.items():
        tau = R * C
        v = V_SOURCE * (1.0 - np.exp(-t / tau))
        results[(r_label, c_label)] = {"R": R, "C": C, "tau": tau, "v": v}

print(f"Computed {len(results)} charging curves.")
```

Computed 9 charging curves.

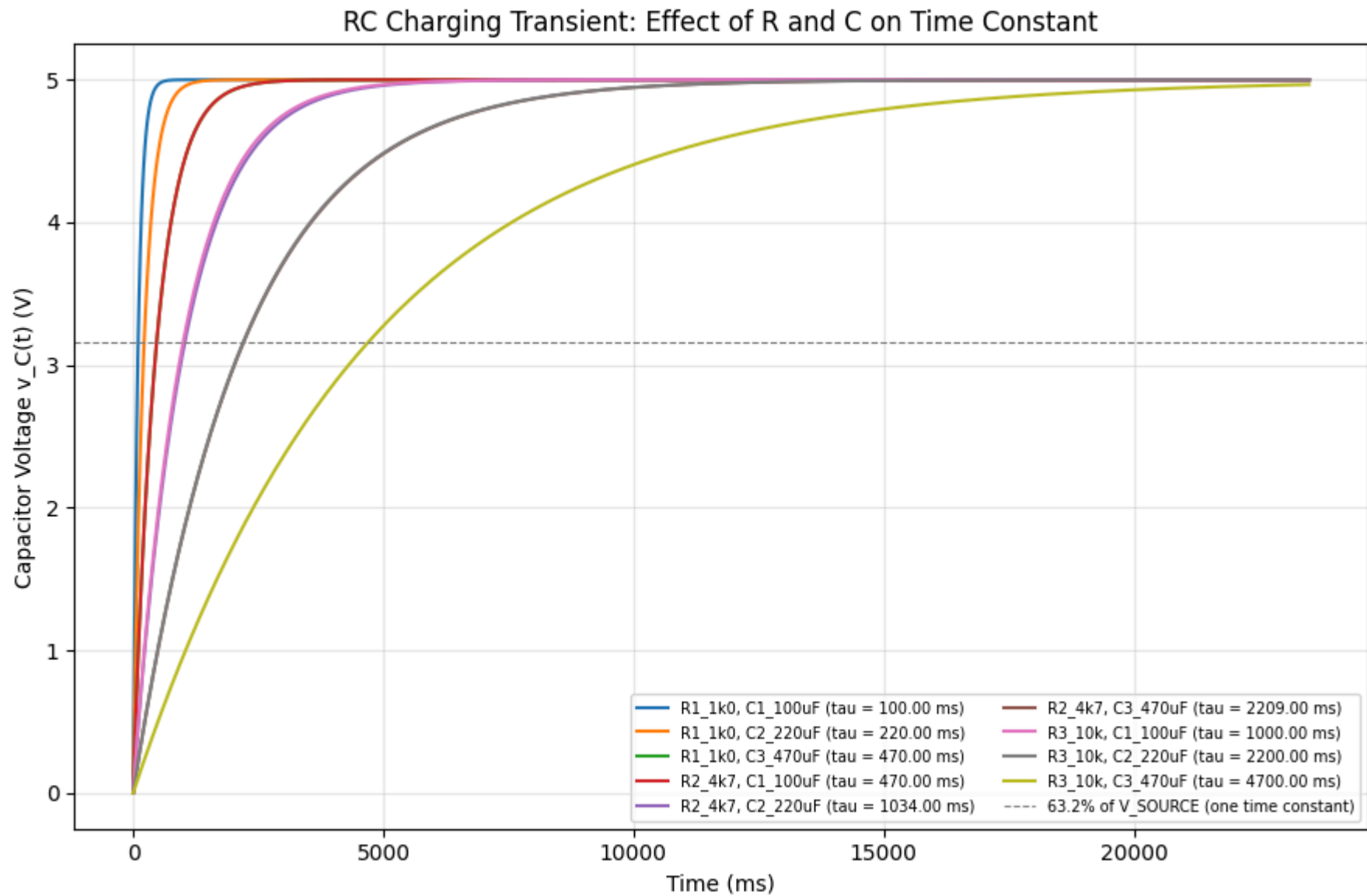
Stage 4: Fig 01 -- overlay plot, absolute time axis

```
In [6]: plt.figure(figsize=(9, 6))
for (r_label, c_label), data in results.items():
    tau_ms = data["tau"] * 1e3
    plt.plot(
        t * 1e3,
        data["v"],
        label=f"{r_label}, {c_label} (tau = {tau_ms:.2f} ms)",
    )

plt.axhline(
    V_SOURCE * 0.632,
```

```
        color="gray",
        linestyle="--",
        linewidth=0.8,
        label="63.2% of V_SOURCE (one time constant)",
    )
plt.xlabel("Time (ms)")
plt.ylabel("Capacitor Voltage v_C(t) (V)")
plt.title("RC Charging Transient: Effect of R and C on Time Constant")
plt.legend(fontsize=7, ncol=2, loc="lower right")
plt.grid(True, alpha=0.3)
plt.tight_layout()

fig01_path = os.path.join(output_dir, "S01_fig01_rc_charging_overlay.png")
plt.savefig(fig01_path, dpi=300)
plt.show()
```



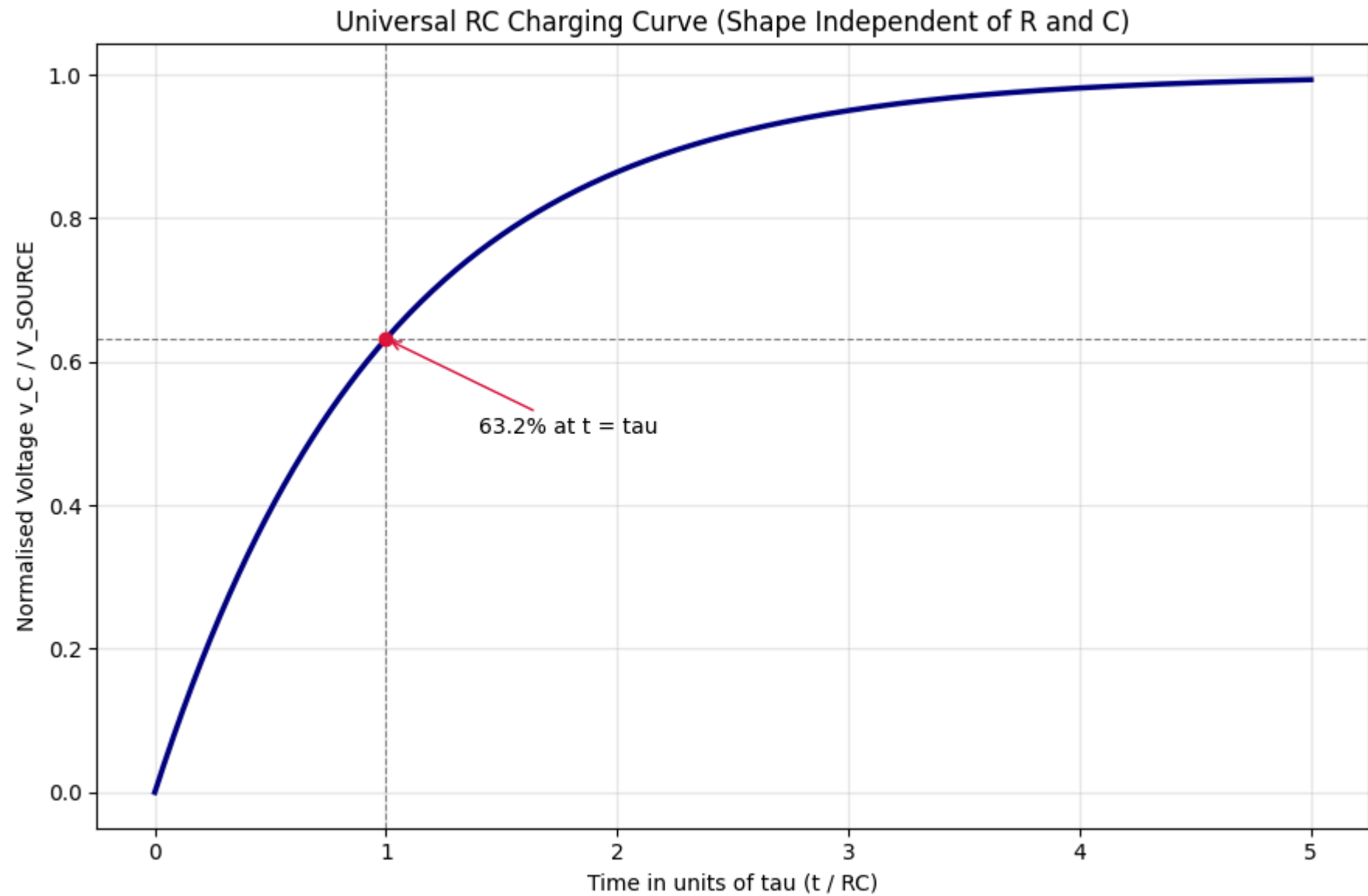
Stage 5: Fig 02 -- normalised curve, time in units of τ

Every (R, C) pair collapses onto this single curve once time is expressed in units of τ . This is the physical content of Module 2: one exponential shape, many circuits.

```
In [7]: plt.figure(figsize=(9, 6))
t_norm = np.linspace(0.0, 5.0, 500)
v_norm = 1.0 - np.exp(-t_norm)

plt.plot(t_norm, v_norm, linewidth=2.5, color="navy")
plt.axvline(1.0, color="gray", linestyle="--", linewidth=0.8)
plt.axhline(0.632, color="gray", linestyle="--", linewidth=0.8)
plt.scatter([1.0], [0.632], color="crimson", zorder=5)
plt.annotate(
    "63.2% at t = tau",
    xy=(1.0, 0.632),
    xytext=(1.4, 0.5),
    arrowprops=dict(arrowstyle="->", color="crimson"),
)
plt.xlabel("Time in units of tau (t / RC)")
plt.ylabel("Normalised Voltage v_C / V_SOURCE")
plt.title("Universal RC Charging Curve (Shape Independent of R and C)")
plt.grid(True, alpha=0.3)
plt.tight_layout()

fig02_path = os.path.join(output_dir, "S02_fig02_normalized_curve.png")
plt.savefig(fig02_path, dpi=300)
plt.show()
```



Stage 6: Export raw data to CSV

```
In [8]: csv_path = os.path.join(output_dir, "S03_data01_rc_charging.csv")
with open(csv_path, "w", newline="") as f:
    writer = csv.writer(f)
    header = ["time_s"] + [f"{r}__{c}_V" for (r, c) in results.keys()]
    writer.writerow(header)
    for i in range(len(t)):
        row = [t[i]] + [results[key]["v"][i] for key in results.keys()]
        writer.writerow(row)

print("Data written to:", csv_path)
```

Data written to: /home/kingsuk/Documents/Kingsuk_LaTeX/AddOnCourse/ClassLecture/Module_01/PythonCode/OUTPUT/S03_data01_rc_charging.csv

Stage 7: Summary report

```
In [9]: summary_path = os.path.join(output_dir, "S04_summary01_report.txt")
with open(summary_path, "w") as f:
    f.write("RC Charging Transient -- Summary Report\n")
    f.write("=" * 60 + "\n")
    f.write(f"Generated : {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")
    f.write(f"V_SOURCE : {V_SOURCE} V\n\n")
    f.write(
        f"{'R label':<10}{'R (ohm)':<12}{'C label':<12}"
        f"{'C (F)':<12}{'tau (ms)':<12}\n"
    )
    f.write("-" * 58 + "\n")
    for (r_label, c_label), data in results.items():
        f.write(
            f"{'r_label':<10}{data['R']:<12.1f}{c_label:<12}"
            f"{'data['C']:<12.2e}{data['tau'] * 1e3:<12.3f}\n"
        )

print("RC charging simulation complete.")
print(f"Figures, data and report saved in: {output_dir}")
```

RC charging simulation complete.

Figures, data and report saved in: /home/kingsuk/Documents/Kingsuk_LaTeX/AddOnCourse/ClassLecture/Module_01/PythonCode/OUTPUT

